

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

PENGANTAR MIKROKONTROLER

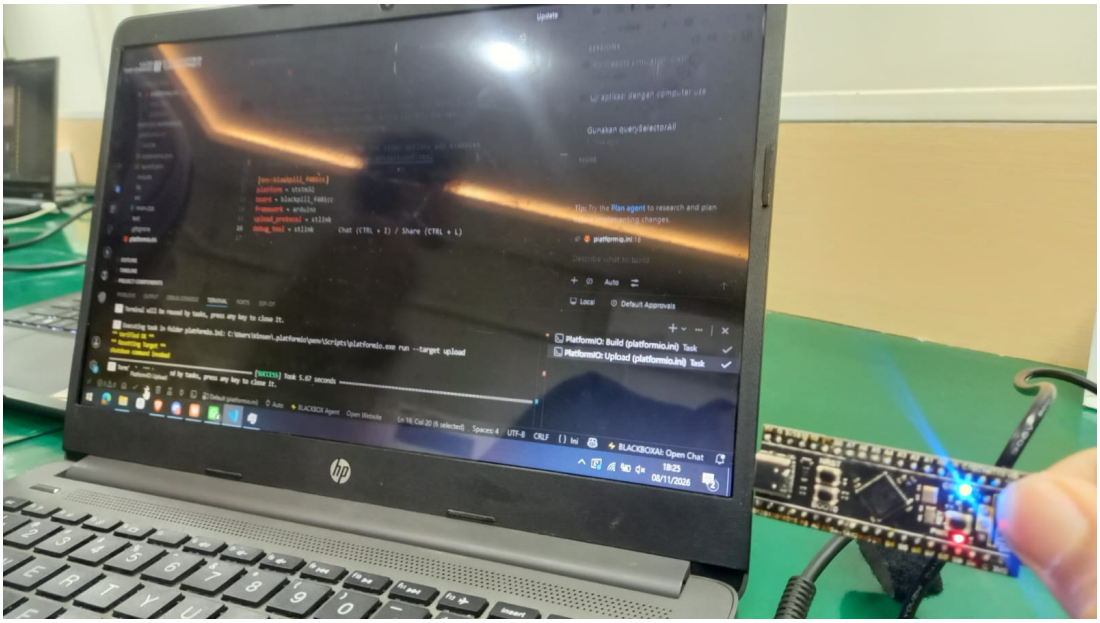
Vinsen Dwi Putra - 5024241094

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Membuat satu proyek baru PlatformIO (`modul1-praktikum`) pada VSCode yang digunakan untuk seluruh rangkaian praktikum.
2. Mengatur berkas `platformio.ini` untuk board `blackpill_f401cc` dengan *framework* `arduino` dan protokol unggah `stlink`, lalu menunggu proses pembaruan indeks proyek selesai.
3. Menghubungkan modul ST-Link V2 ke STM32 Blackpill melalui pin `SWDIO`, `SWCLK`, `GND`, dan `3.3V`, kemudian menyambungkan ST-Link ke port USB laptop.
4. Menulis kode program *Blink* pada `src/main.cpp` untuk mengatur pin LED internal `PC13` sebagai keluaran digital dengan jeda waktu 500 ms.
5. Melakukan **Build** dan **Upload** ke board, lalu mengamati kedipan LED internal `PC13` beserta tampilan layar VSCode sebagai bukti pelaksanaan percobaan (Gambar 1).

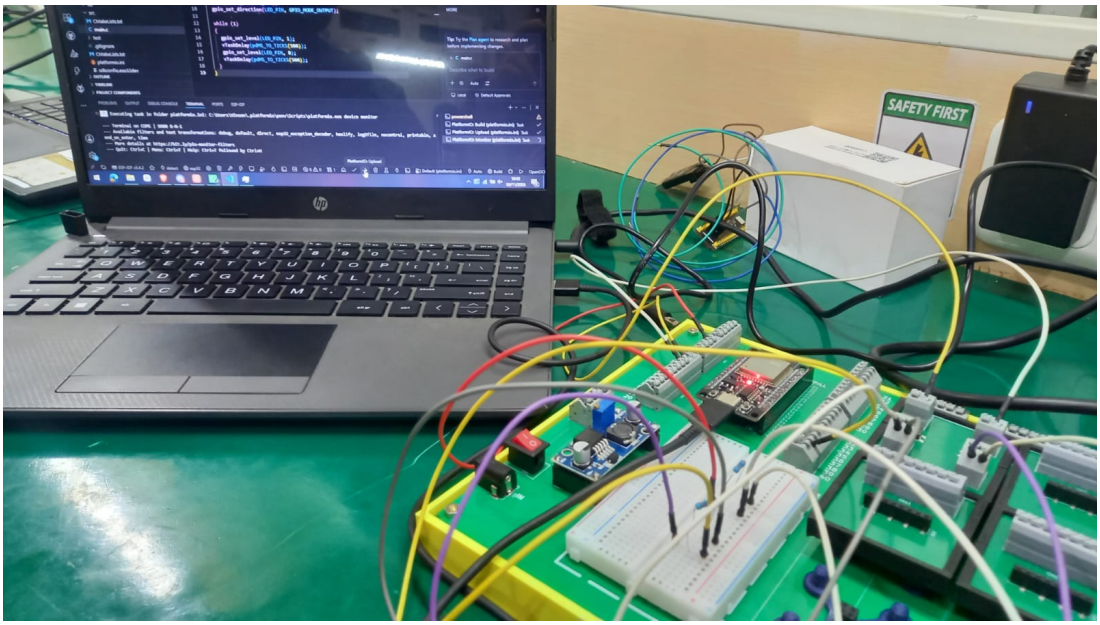


Gambar 1: Bukti tampilan layar laptop (kode dan proses unggah PlatformIO) pada Percobaan 1 serta LED internal STM32 yang menyala.

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Mengubah isi `platformio.ini` menjadi board `esp32dev` dengan *framework* `espidf`, kemudian menunggu proses inisialisasi komponen ESP-IDF selesai.
2. Menghapus berkas `src/main.cpp` dan membuat berkas sumber baru bernama `src/main.c`.
3. Menghubungkan ESP32 DevKit v1 ke laptop menggunakan kabel USB dan memastikan port serial terbaca oleh sistem.
4. Menulis program *Blink* berbasis FreeRTOS di dalam fungsi `app_main()` menggunakan `GPIO_NUM_2` dan fungsi tunda `vTaskDelay(pdMS_TO_TICKS(500))`.

5. Melakukan **Build** dan **Upload**, lalu mengamati kedipan LED internal ESP32 serta mendokumentasikan layar kode dan board yang menyala (Gambar 2).



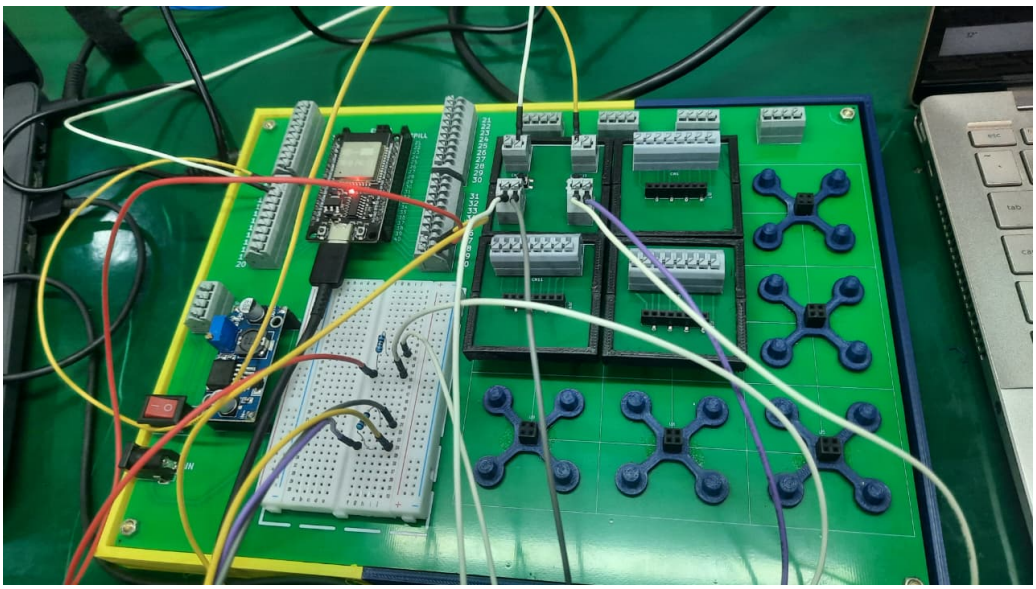
Gambar 2: Bukti tampilan layar laptop (kode ESP-IDF dan hasil kompilasi) pada Percobaan 2 serta LED internal ESP32 yang menyala.

1.3 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

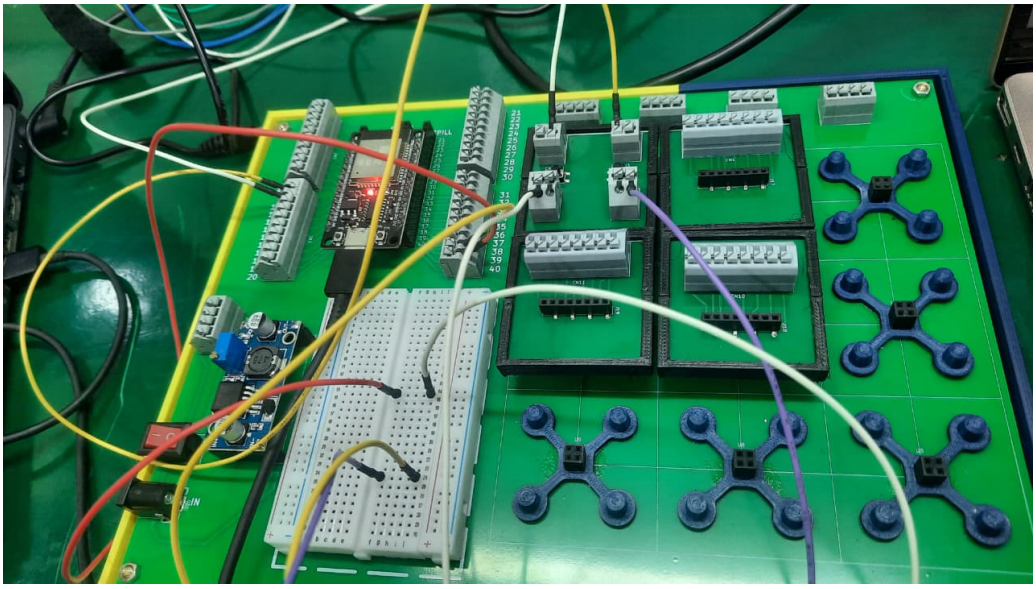
1. Mengembalikan konfigurasi `platformio.ini` ke *framework* `arduino` untuk board `esp32dev`, serta mengganti berkas `src/main.c` kembali menjadi `src/main.cpp`.
2. Merangkai dua *pushbutton* pada *breadboard*: tombol pertama ke `GPIO 32` dengan resistor *pull-up* eksternal 10 kΩ ke $3{,}3$ V, dan tombol kedua ke `GPIO 33` dengan resistor *pull-down* eksternal 10 kΩ ke `GND` (Gambar 3).
3. Mengunggah program pembacaan tombol eksternal (`INPUT`), membuka *Serial Monitor* (115200 baud), lalu menguji penekanan kedua tombol secara bergantian.
4. Melepas kedua resistor eksternal 10 kΩ dari *breadboard* sehingga hanya menyisakan sambungan tombol ke GPIO dan tegangan referensi (Gambar 4).
5. Mengubah mode pin pada kode menjadi `INPUT_PULLUP` (`GPIO 32`) dan `INPUT_PULLDOWN` (`GPIO 33`), mengunggah ulang kode, lalu mengamati keluaran pada *Serial Monitor* (Gambar 5).

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

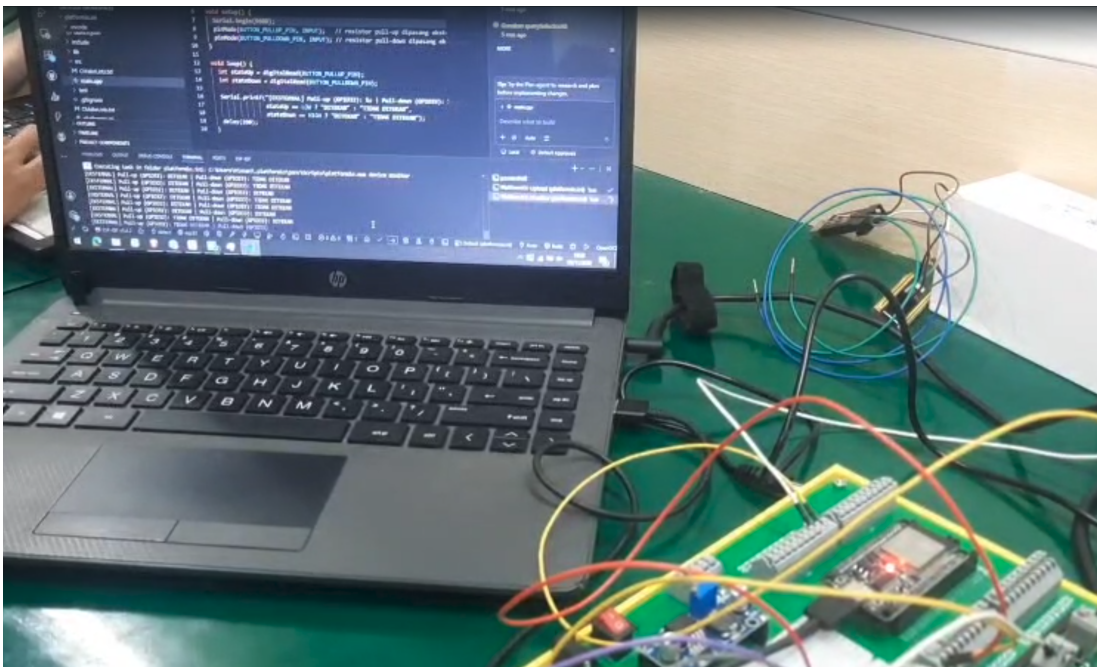
1. Merangkai satu *pushbutton* pada *breadboard* yang terhubung antara pin `GPIO 25` ESP32 dan jalur `GND` (Gambar 6).



Gambar 3: Rangkaian konfigurasi *pull-up* dan *pull-down* pada Percobaan 3 menggunakan resistor eksternal $10\ \text{k}\Omega$.

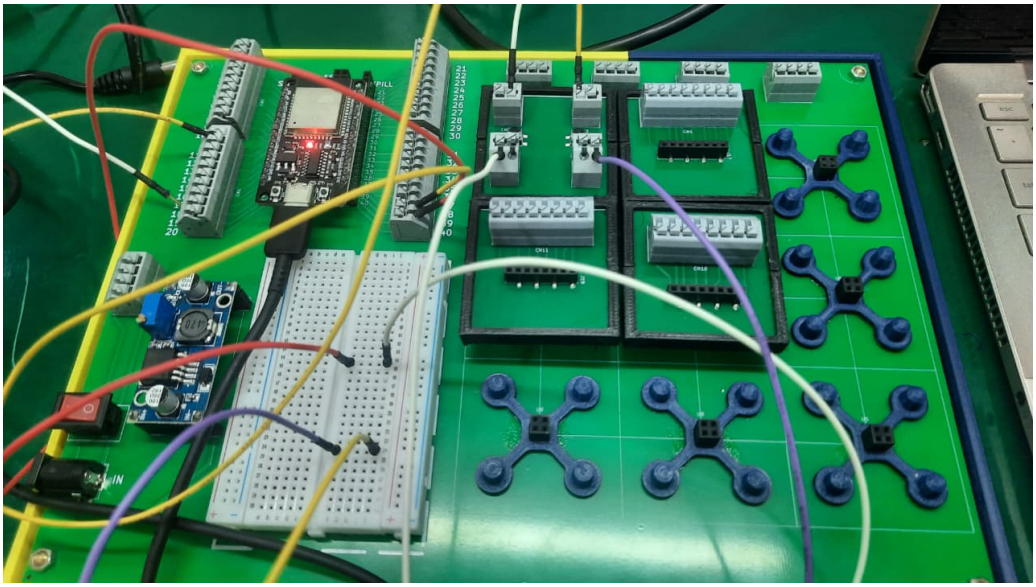


Gambar 4: Rangkaian konfigurasi *pull-up* dan *pull-down* pada Percobaan 3 tanpa resistor eksternal (menggunakan resistor internal).

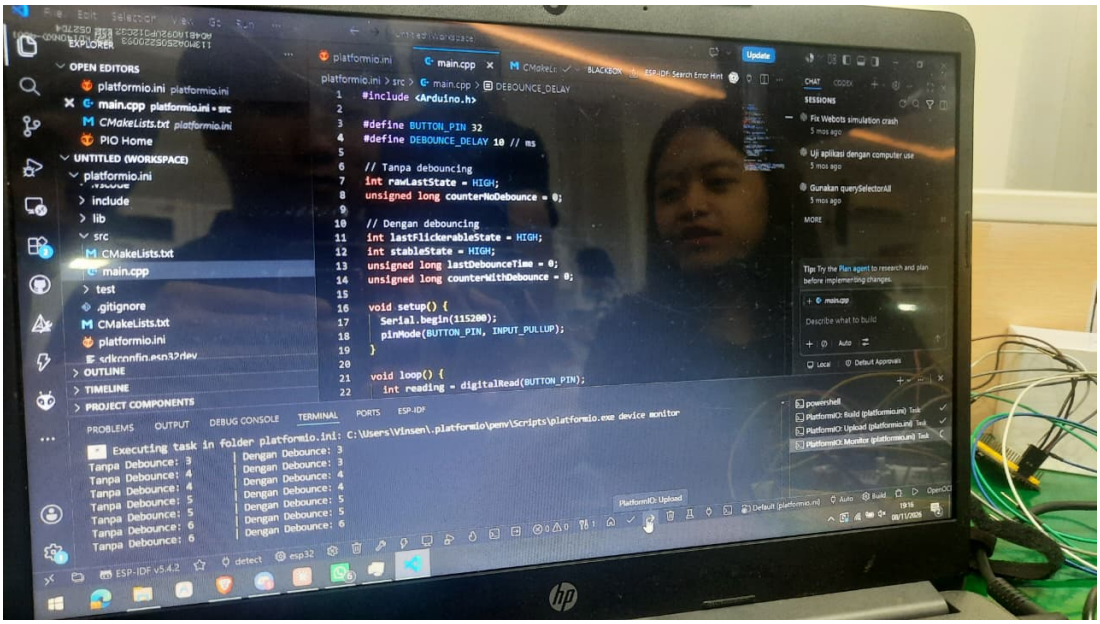


Gambar 5: Tampilan layar laptop dan hasil pembacaan logika tombol pada *Serial Monitor* untuk Percobaan 3.

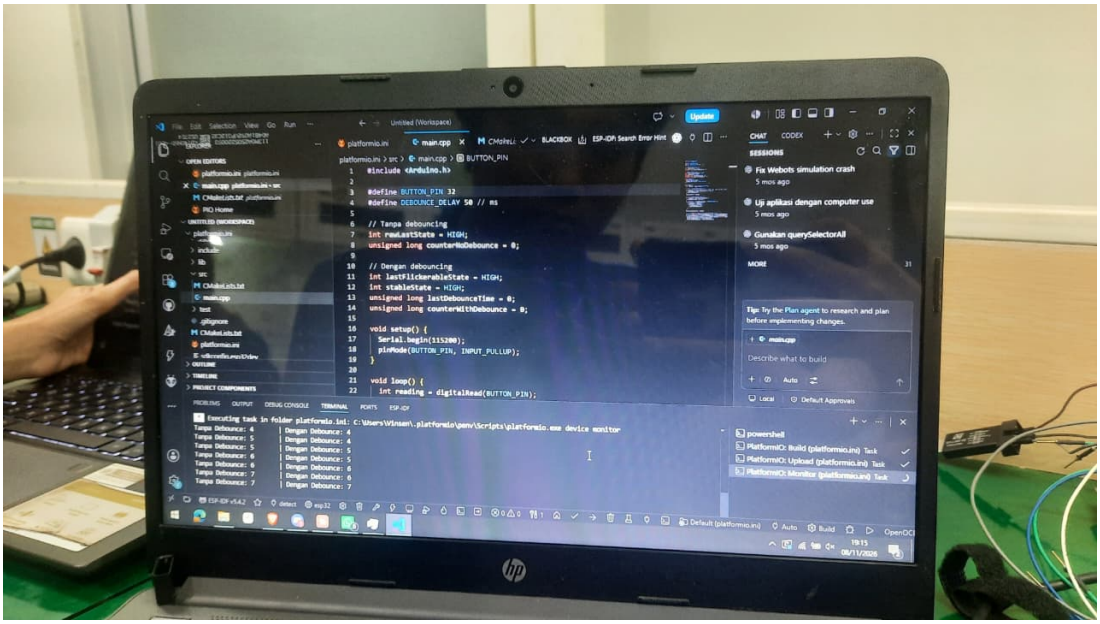
2. Menulis program perbandingan `counterNoDebounce` dan `counterWithDebounce` dengan mode `INPUT_PULLUP` serta nilai awal `DEBOUNCE_DELAY` sebesar 50 ms.
3. Mengunggah program ke ESP32 dan membuka *Serial Monitor* pada kecepatan 115200 baud.
4. Menekan tombol berulang kali dengan variasi tekanan (cepat dan tegas maupun perlahan) untuk membandingkan selisih kedua pencacah pada nilai tunda 50 ms (Gambar 8).
5. Mengubah parameter `DEBOUNCE_DELAY` menjadi 10 ms (Gambar 7) dan 200 ms (Gambar 9), lalu mengunggah ulang program untuk membandingkan kestabilan dan kecepatan respons tombol pada masing-masing pengujian.



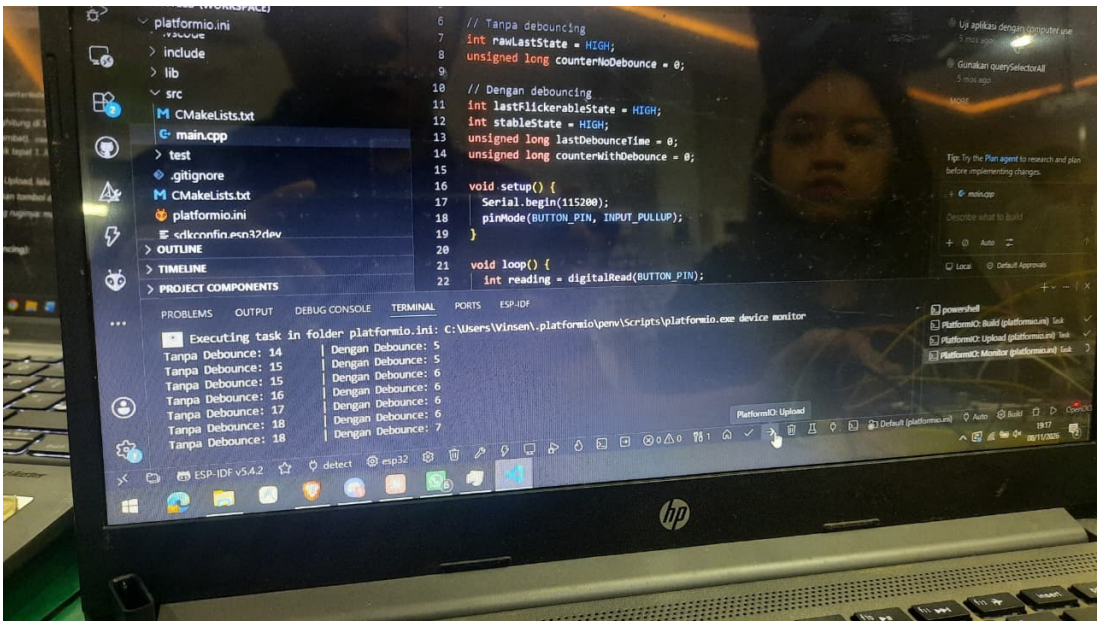
Gambar 6: Rangkaian tombol uji *debouncing* pada `GPIO 25` ESP32 pada Percobaan 4.



Gambar 7: Hasil pembacaan pencacah pada *Serial Monitor* dengan pengaturan `DEBOUNCE_DELAY` sebesar 10 ms.



Gambar 8: Hasil pembacaan pencacah pada *Serial Monitor* dengan pengaturan `DEBOUNCE_DELAY` sebesar 50 ms.



Gambar 9: Hasil pembacaan pencacah pada *Serial Monitor* dengan pengaturan `DEBOUNCE_DELAY` sebesar 200 ms.

1.5 Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

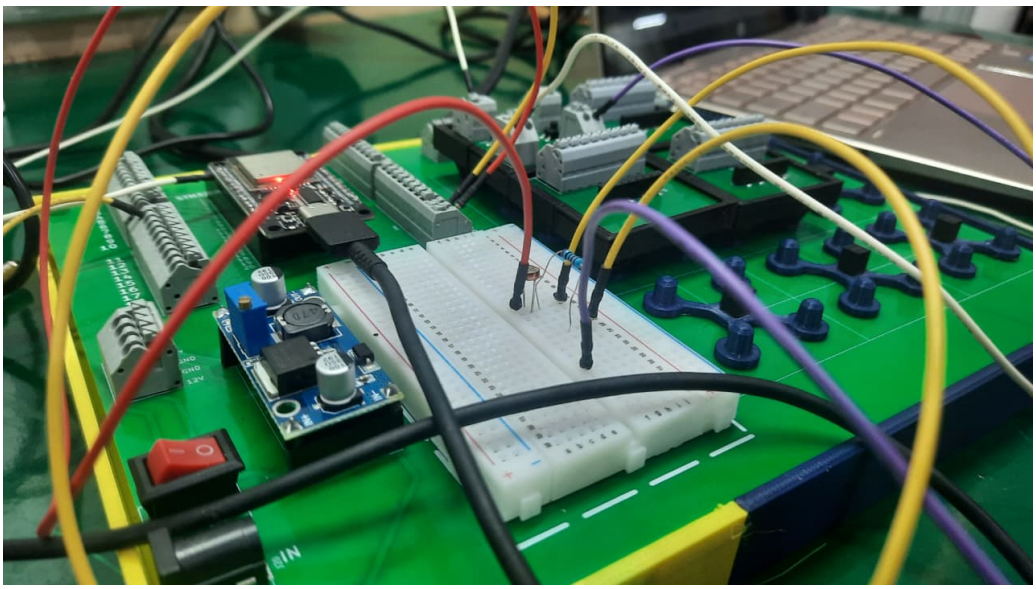
1. Merangkai pembagi tegangan menggunakan LDR dan resistor 10 kΩ, dengan satu kaki LDR ke $3{,}3$ V, satu kaki resistor ke `GND`, dan titik tengah keduanya dihubungkan ke `GPIO 34` ESP32 (Gambar 10).

2. Menulis kode program pada `src/main.cpp` untuk membaca nilai mentah ADC 12-bit melalui `analogRead(34)` dan mengonversinya ke dalam satuan tegangan volt.

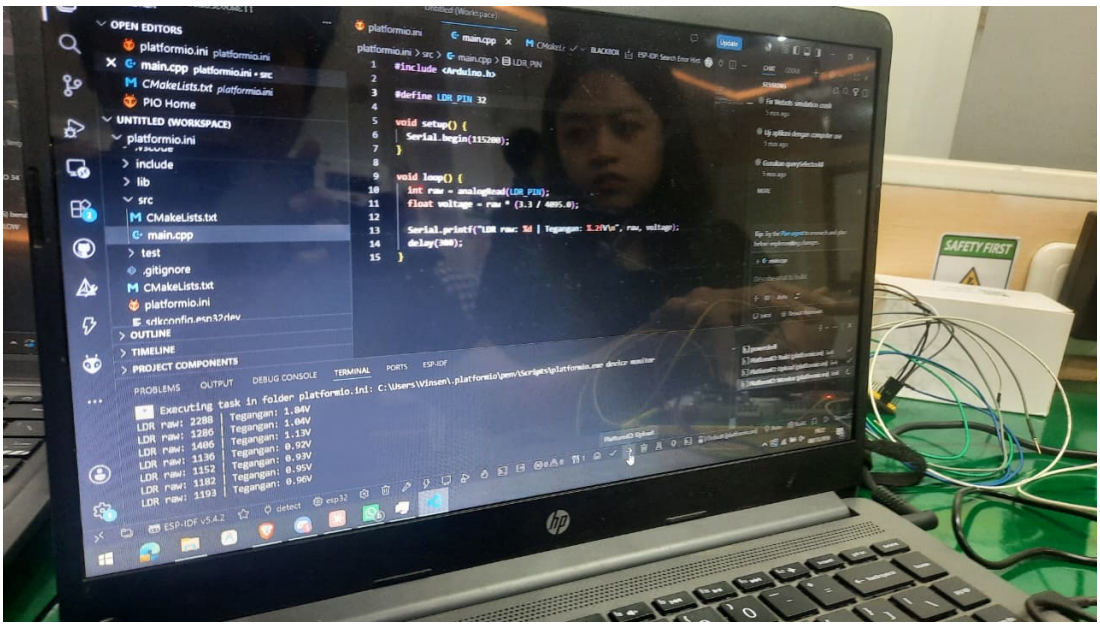
3. Mengunggah program ke ESP32 dan membuka *Serial Monitor* (115200 baud).

4. Memberikan variasi intensitas cahaya pada sensor dengan cara menutup permukaan LDR menggunakan tangan (kondisi gelap) dan menyinarinya dengan cahaya (kondisi terang).

5. Mengamati perubahan nilai ADC (0–4095) beserta konversi tegangannya pada layar *Serial Monitor* (Gambar 11).



Gambar 10: Rangkaian pembagi tegangan sensor LDR dan resistor 10 kΩ pada `GPIO 34` ESP32.



Gambar 11: Tampilan layar laptop yang menunjukkan perubahan nilai mentah ADC dan tegangan pembacaan LDR pada Percobaan 5.

2 Analisis Hasil Percobaan

2.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Berdasarkan hasil pengunggahan program melalui ST-Link V2, LED indikator bawaan pada pin `PC13` papan STM32 Blackpill berkedip secara periodik setiap 500 ms. Hasil ini sesuai dengan teori arsitektur board STM32F401CCU6 yang menerapkan konfigurasi *active-low* pada LED internalnya, sehingga logika `LOW` menyalakan LED dan logika `HIGH` memadamkannya. Faktor keberhasilan pada tahap ini sangat bergantung pada ketepatan pemasangan empat pin antarmuka SWD (`SWDIO`, `SWCLK`, `GND`, `3.3V`) dan jeda sinkronisasi indeks PlatformIO sebelum proses kompilasi dijalankan.

2.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Program *Blink* yang dikompilasi menggunakan kerangka kerja ESP-IDF berhasil menyalakan dan memadamkan LED internal ESP32 pada `GPIO_NUM_2` dengan interval 500 ms. Berbeda dengan siklus sekuensial Arduino, eksekusi program dimulai dari fungsi `app_main()` yang beroperasi sebagai sebuah *task* FreeRTOS, di mana fungsi `vTaskDelay(pdMS_TO_TICKS(500))` memberikan penundaan berbasis *tick* RTOS yang bersifat *non-blocking*. Hal ini membuktikan teori bahwa ESP-IDF memberikan kendali manajemen sumber daya tingkat rendah secara langsung di atas sistem operasi waktu nyata.

2.3 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

Pengujian pada `GPIO 32` (*pull-up*) dan `GPIO 33` (*pull-down*) menghasilkan perilaku logika yang saling berkebalikan dan konsisten antara rangkaian resistor eksternal maupun internal. Konfigurasi *pull-up* mempertahankan logika default `HIGH` saat tombol dilepas dan berubah menjadi `LOW` saat ditekan, sedangkan konfigurasi *pull-down* menghasilkan logika default `LOW` saat dilepas dan `HIGH` saat ditekan. Kesamaan hasil antara kedua metode membuktikan bahwa resistor internal ESP32 efektif mencegah kondisi *floating*, dengan catatan resistor eksternal wajib dilepas sepenuhnya saat menguji mode internal agar tidak terjadi pembagian tegangan yang mengganggu pembacaan logika.

2.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Hasil pengamatan menunjukkan bahwa nilai `counterNoDebounce` sering bertambah lebih dari satu hitungan dalam satu kali penekanan fisik, sedangkan `counterWithDebounce` bertambah tepat satu hitungan. Fenomena ini mengonfirmasi teori pentalan kontak mekanis (*bouncing*) yang memicu osilasi sinyal cepat selama beberapa milidetik, di mana faktor cara menekan tombol (tekanan pelan atau ragu-ragu) memperpanjang durasi osilasi mekanis tersebut.

Evaluasi variasi parameter `DEBOUNCE_DELAY` memperlihatkan adanya pertukaran (*trade-off*) antara ketahanan terhadap derau dan kecepatan respons sistem. Pada nilai 10 ms, getaran tombol yang berlangsung lebih lama masih sesekali lolos sebagai hitungan ganda, sedangkan pada nilai 200 ms sinyal bebas dari derau tetapi sistem terasa lambat saat tombol ditekan berulang secara cepat. Parameter 50 ms terbukti paling optimal dalam meredam pantulan mekanik tanpa mengorbankan responsivitas input.

2.5 Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

Pembacaan pin `GPIO 34` menunjukkan perubahan nilai digital mentah dan konversi tegangan secara kontinu sesuai intensitas cahaya pada permukaan LDR. Saat sensor ditutup rapat (gelap), resistansi LDR meningkat tajam sehingga pembagi tegangan menghasilkan nilai rendah mendekati 0 V, sedangkan saat disinari cahaya terang, resistansi LDR turun drastis sehingga tegangan mendekati $3{,}3$ V dengan nilai ADC mendekati 4095. Faktor yang memengaruhi variasi nilai pembacaan di lapangan meliputi kebocoran cahaya ruangan di sela jari saat menutup sensor serta toleransi nilai komponen resistor 10 kΩ pada rangkaian pembagi tegangan.

3 Hasil Tugas Modul

3.1 Rancangan Simulasi Matrix Pushbutton 3x3 dan Matrix LED 3x3

Simulasi dirancang menggunakan mikrokontroler ESP32 pada platform Wokwi dengan menghubungkan *matrix pushbutton* 3×3 dan *matrix LED* 3×3 pada pin GPIO terpisah. Pembacaan tombol menerapkan metode pemindaian baris (*row scanning*) aktif `LOW` dan konfigurasi `INPUT_PULLUP` pada pin kolom, disertai algoritma *software debouncing* independen berbasis `millis()` untuk setiap koordinat matriks $[r][c]$. Ketika penekanan tombol pada posisi (r, c) tervalidasi stabil melampaui 50 ms, status logika LED pada posisi (r, c) yang sama akan diubah (*toggle*) dan ditampilkan melalui teknik *multiplexing* cepat.

Berikut adalah kode program yang diimplementasikan pada simulasi ESP32:

```
#include <Arduino.h>

const int btnRowPins[3] = {13, 12, 14};
const int btnColPins[3] = {27, 26, 25};
const int ledRowPins[3] = {33, 32, 15}; // Anoda LED (+ resistor)
const int ledColPins[3] = {4, 16, 17};   // Katoda LED

const unsigned long DEBOUNCE_DELAY = 50;
int lastFlickerableState[3][3];
int stableState[3][3];
unsigned long lastDebounceTime[3][3];
bool ledMatrixState[3][3];

void setup() {
  for (int i = 0; i < 3; i++) {
    pinMode(btnRowPins[i], OUTPUT);
    digitalWrite(btnRowPins[i], HIGH);
    pinMode(btnColPins[i], INPUT_PULLUP);

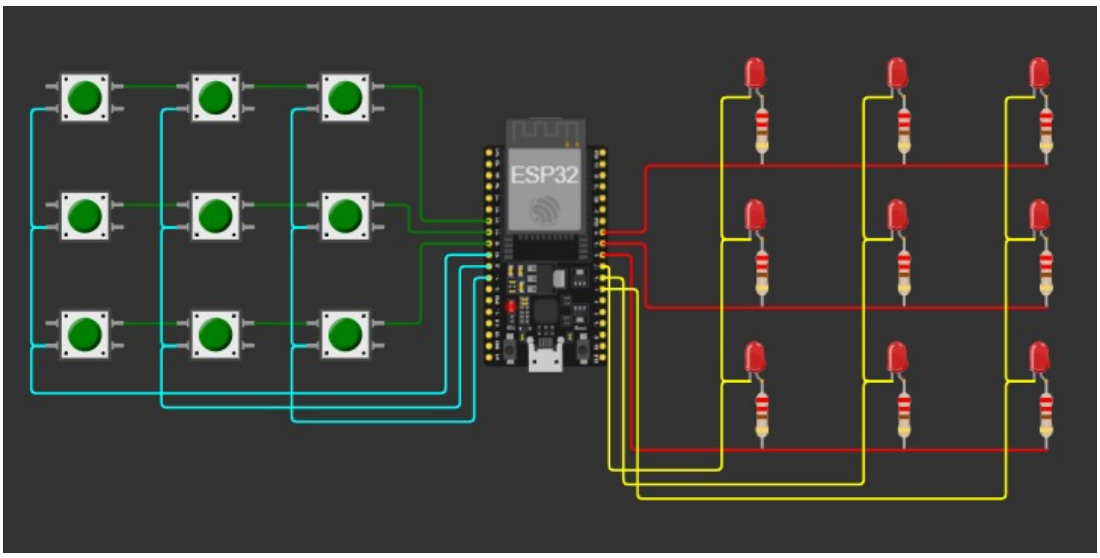
    pinMode(ledRowPins[i], OUTPUT);
    digitalWrite(ledRowPins[i], LOW);
    pinMode(ledColPins[i], OUTPUT);
    digitalWrite(ledColPins[i], HIGH);

    for (int j = 0; j < 3; j++) {
      lastFlickerableState[i][j] = HIGH;
      stableState[i][j] = HIGH;
      lastDebounceTime[i][j] = 0;
      ledMatrixState[i][j] = false;
    }
  }
}
```

```
void scanButtons() {
  for (int r = 0; r < 3; r++) {
    digitalWrite(btnRowPins[r], LOW);
    for (int c = 0; c < 3; c++) {
      int reading = digitalRead(btnColPins[c]);
      if (reading != lastFlickerableState[r][c]) {
        lastDebounceTime[r][c] = millis();
      }
      if ((millis() - lastDebounceTime[r][c]) > DEBOUNCE_DELAY) {
        if (reading != stableState[r][c]) {
          stableState[r][c] = reading;
          if (stableState[r][c] == LOW) {
            ledMatrixState[r][c] = !ledMatrixState[r][c];
          }
        }
      }
      lastFlickerableState[r][c] = reading;
    }
    digitalWrite(btnRowPins[r], HIGH);
  }
}

void refreshLEDs() {
  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      digitalWrite(ledColPins[c], ledMatrixState[r][c] ? LOW : HIGH);
    }
    digitalWrite(ledRowPins[r], HIGH);
    delayMicroseconds(1000);
    digitalWrite(ledRowPins[r], LOW);
  }
}

void loop() {
  scanButtons();
  refreshLEDs();
}
```



Gambar 12: Tangkapan layar hasil simulasi pengendalian Matrix LED 3x3 menggunakan Matrix Pushbutton 3x3 dengan algoritma *debouncing* pada ESP32.

3.2 Jawaban Pertanyaan Analisis

1. **Urgensi Debouncing pada Matrix Pushbutton:** Fenomena *bouncing* berasal dari sifat elastisitas pelat kontak logam mekanis di dalam tombol yang memantul secara mikro selama rentang 1 hingga 50 ms saat ditekan atau dilepas. Meskipun hanya satu tombol yang ditekan pada satu waktu, mikrokontroler memindai matriks dalam hitungan mikrodetik sehingga pantulan satu tombol saja akan terbaca sebagai beberapa kali transisi penekanan beruntun yang menyebabkan status nyala LED berubah secara tidak menentu.

2. **Konfigurasi Pull-up dan Bahaya Kondisi Floating:** Rangkaian menggunakan konfigurasi *pull-up* internal (`INPUT_PULLUP`) pada pin kolom dengan pemindaian baris aktif `LOW`. Saat tombol tidak ditekan, pin kolom ditarik ke rel tegangan $3{,}3$ V sehingga terbaca berlogika **HIGH**. Pin masukan tidak boleh dibiarkan mengambang (*floating*) karena impedansi masukan gerbang logika CMOS yang sangat tinggi akan menyerap derau elektrostatik lingkungan sekitar, memicu fluktuasi logika acak antara `HIGH` dan `LOW` yang menyebabkan penyalaan LED palsu.

3. **Perbedaan Input Digital (Tombol) dan Input Analog (ADC/LDR):** Pada mode *digital input*, rangkaian komparator *Schmitt trigger* internal GPIO hanya membandingkan tegangan masukan terhadap dua ambang batas logika untuk menghasilkan dua keadaan biner diskret (`HIGH` atau `LOW`). Sebaliknya, pada mode *analog input*, tegangan kontinu yang masuk dicuplik dan dikuantisasi oleh periferal ADC 12-bit ESP32 ke dalam $2^{12} = 4096$ tingkatan nilai diskret (0–4095) pada rentang 0–$3{,}3$ V, sehingga mampu merepresentasikan variasi intensitas cahaya secara bertingkat dan kuantitatif.

4 Kesimpulan

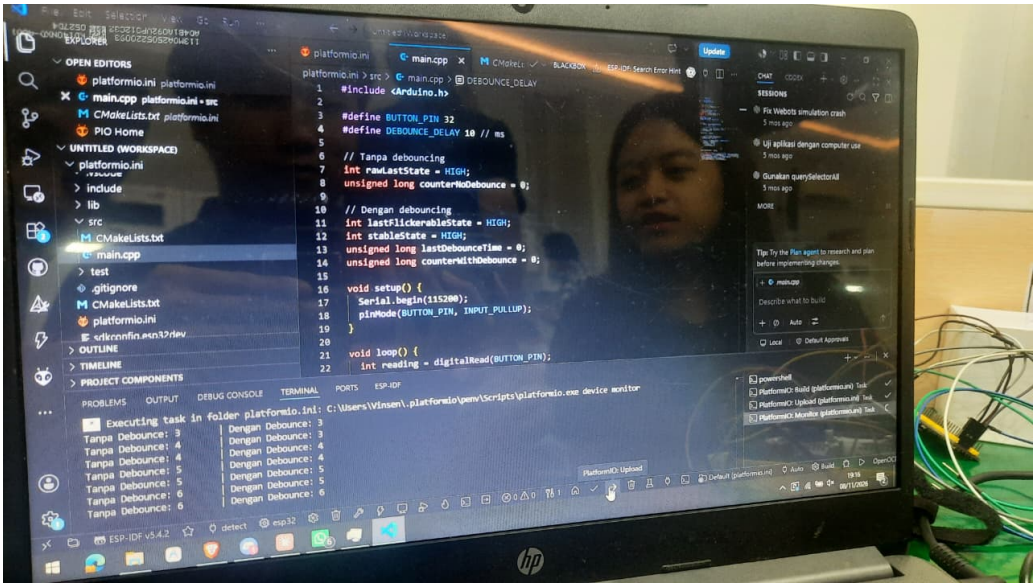
Berdasarkan seluruh rangkaian praktikum Modul 1, tujuan pembelajaran telah tercapai melalui instalasi serta konfigurasi PlatformIO pada VSCode untuk memprogram arsitektur STM32 Blackpill dan ESP32 secara terpadu. Perbedaan karakteristik kedua papan beserta kerangka kerjanya terverifikasi

sesuai dengan teori, di mana STM32 Blackpill memerlukan pemrogram eksternal ST-Link melalui antarmuka SWD dengan abstraksi praktis STM32duino, sedangkan ESP32 dapat diprogram langsung via USB dengan kendali *native* berbasis FreeRTOS pada ESP-IDF maupun abstraksi cepat pada Arduino.

Seluruh pengujian mode akses GPIO pada praktikum juga memberikan hasil yang konsisten dengan landasan teori elektronika digital dan analog. Penerapan resistor *pull-up* dan *pull-down* (eksternal maupun internal) terbukti mencegah kondisi *floating* pada masukan tombol, sementara algoritma *software debouncing* dengan parameter optimal 50 ms mampu meredam osilasi mekanik tombol tanpa menurunkan responsivitas sistem. Selain itu, penggunaan periferal ADC 12-bit pada ESP32 berhasil mengonversi perubahan resistansi sensor LDR menjadi rentang nilai diskret 0–4095 sebagai representasi variasi besaran analog lingkungan secara akurat.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 13: Dokumentasi praktikan